시스템 프로그래밍

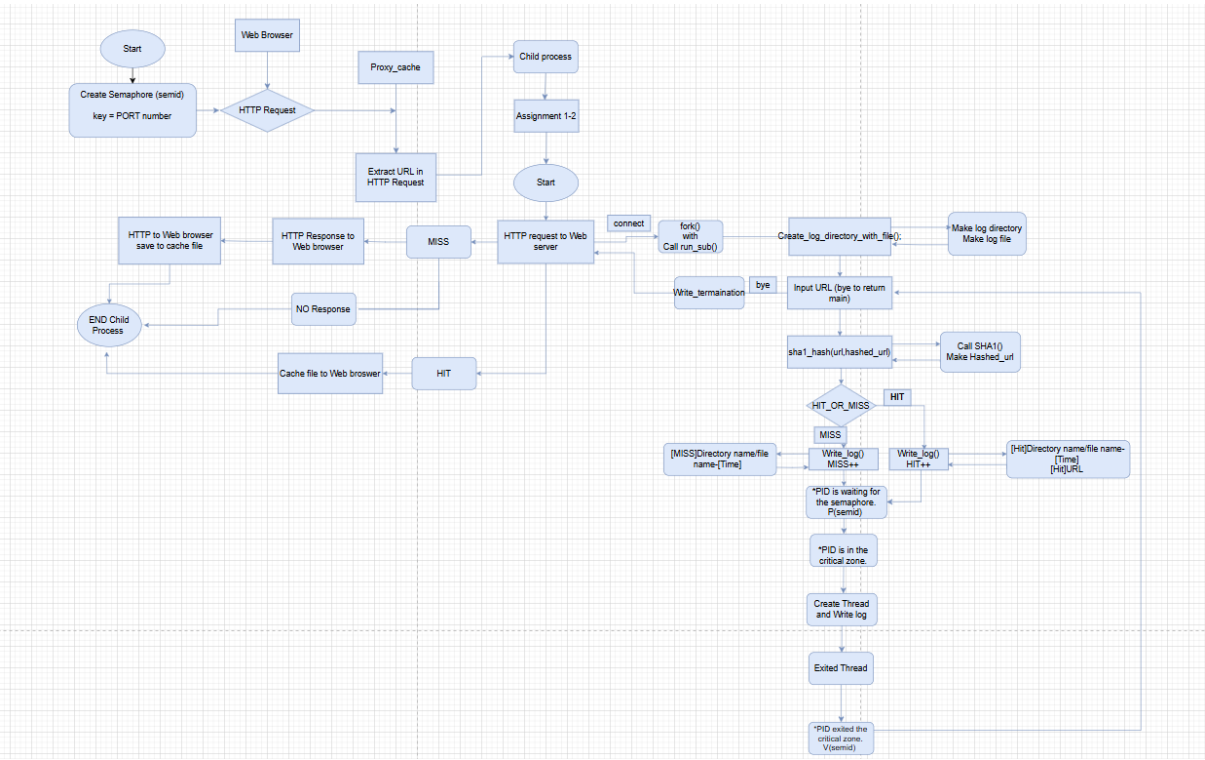
Proxy 3-2

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

Introduction

마지막 과제인 Proxy 3-2 에서는, 이전에한 Proxy 3-1 과제에 Critical section 내에서 log를 기록하는 thread를 추가하는 과제이다. (이때 쓰레드란 프로세스 내에서 실제로 작업을 수행하는 주체를 의미하는데, 모든 프로세스는 한 개 이상의 쓰레드가 존재하여 작업을 수행하게 된다.) 그리고 할당된 thread내에서 어떤 자식 프로세스가 스레드를 생성한지를 알 수 있게끔, 터미널상에서 자식 프로세스와 해당 프로세스에서 생성된 스레드의 ID를 출력한다. 또한 스레드가 종료될 때는 해당 id를 가진 스레드가 종료되는 것을 알려주기 위해서, 터미널에 스레드 종료 메시지를 출력한다. 또한 이번 과제에서는 캐시 락 경쟁 문제를 고려하지 않으므로, 캐시 파일을 동시에 접근해서 생기는 문제는 고려하지 않아도 됩니다.

Flow chart



Pseudo code

```
main:
    initialize server socket (bind, listen)
    create semaphore with key = PORT

    loop forever:
        accept client connection
        fork process
        if child process:
            handle_client()
            exit
        else:
            close client socket (in parent)
-------------------------------------------------
handle_client():
    read HTTP request from client
    parse URL
    hash URL to create hashed_path

    if cache exists for hashed_path:
        send cached response to client
        Write_log_in_file(url, hashed_url, "Hit")
    else:
        connect to original web server
        fetch response
```

```
            cache response
            send to client
            Write_log_in_file(url, hashed_url, "Miss")
        close client connection
-------------------------------------------------
Write_log_in_file(url, hashed_url, type):
        sleep
        print "*PID is waiting for the semaphore"
        P(semid)   // acquire semaphore
        print "*PID is in the critical zone"
        Create Thread
        Print "*PID# getpid() create the *TID# pthread_ID"
        open logfile in append mode
        get current time
        if type == "Hit":
            write [Hit] log
        else if type == "Miss":
            write [Miss] log
       Exited Thread with print Exited message
        sleep
        print "*PID exited the critical zone"
        close logfile
        V(semid)   // release semaphore
-------------------------------------------------
```

결과화면

```
.kw2021202089@ubuntu:~$ ./proxy_cache
*PID# 5319 is waiting for the semaphore.
*PID# 5333 is waiting for the semaphore.
*PID# 5338 is waiting for the semaphore.
*PID# 5319 is in the critical zone.
*PID# 5350 is waiting for the semaphore.
*PID 5319 create the *TID# 139958355257088.
*TID# 139958355257088 is exited.
*PID# 5319 exited the critical zone.
*PID# 5333 is in the critical zone.
*PID 5333 create the *TID# 139958355257088.
*TID# 139958355257088 is exited.
*PID# 5333 exited the critical zone.
*PID# 5338 is in the critical zone.
*PID 5338 create the *TID# 139958355257088.
*TID# 139958355257088 is exited.
*PID# 5338 exited the critical zone.
*PID# 5350 is in the critical zone.
*PID 5350 create the *TID# 139958355257088.
*TID# 139958355257088 is exited.
*PID# 5350 exited the critical zone.
^C
```

⇨ 이전에 Proxy 3-1 에서 구현한 Semaphore 동기화가 마찬가지로 잘 이루어짐을 알 수 있습니다.

⇨ 또한 Critical section에 진입한 해당 프로세스에서 thread가 형성되어서, log file을 작성하고, 스레드가 종료된 뒤 Critical section또한 해제됨을 알 수 있습니다.

```
kw2021202089@ubuntu:~$ cat ./logfile/logfile.txt
[Miss] ServerPID : 3606 | http://httpbin.org/get - [2025/5/31, 03:59:42]
[Miss] ServerPID : 3618 | http://textfiles.com/ - [2025/5/31, 03:59:49]
[Miss] ServerPID : 3629 | http://www.catb.org/jargon/ - [2025/5/31, 03:59:56]
[Miss] ServerPID : 3749 | http://neverssl.com/ - [2025/5/31, 04:00:21]
**SERVER** [Terminated] run time: 115 sec. #sub process: 23
```

⇨ 로그 파일도 이전 Proxy 3-1의 기능과 동일하게 기록됨을 알 수 있습니다.

고찰

마지막 Proxy 3-2 과제는 Critical Section 내에서 로그를 기록하는 thread를 추가하는 것이었습니다. 기존 3-1에서 구현한 log 기록 함수를 수정하지 않고, 쓰레드로 분리하는 과정에서 pthread의 사용법을 익힐 수 있었습니다.

특히 pthread_create, pthread_join 등의 함수와 쓰레드 간 자원 공유, 동기화 개념을 자연스럽게 이해하게 되었습니다.

로그 기록 도중 세마포어로 보호되는 영역에서 thread를 생성해 로그를 Proxy 3-1과 마찬가지로 남기는 것이 중요했으며, 이를 위해 로그 출력 시점과 thread 종료 시점까지 메시지를 정확히 출력하도록 구현하였습니다.

처음에는 TID 출력이 의도한 대로 되지 않아 syscall(SYS_gettid)를 사용하는 등의 시행착오도 있었습니다.

최종적으로 thread가 정상적으로 생성 및 종료되며 로그를 남기는 흐름을 구현하면서, 병렬 처리와 동기화의 중요성을 체감할 수 있었습니다. 또한 이론 시간에 들었을 때는 프로세스와 스레드간에 정확한 관계가 무엇인지 헷갈렸는데, 이번 과제를 통해서, 스레드는 프로세스 내부에서 실행되는 작업 단위임을 알 수 있었습니다.

이렇게 이번학기 시스템프로그래밍 과제를 모두 마쳤는데, 처음에는 Proxy 서버에 대해서 배웠을 때는 막연한 느낌이 있었고, 또한 리눅스 사용 또한 자주 해보지 않아서 헷갈리는 점이 많았는데, 주기적인 과제를 통해서, 수업시간 내용을 계속해서 상기하니까 효율적으로 학습할 수 있었던 것 같습니다.

이번학기 고생하셨습니다.

Reference